



FASTIFY CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & INSTANCE

Installation

```
npm install fastify
import Fastify from 'fastify'
const fastify = Fastify({ logger: true })
await fastify.listen({ port: 3000, host: '0.0.0.0' })
fastify.log.info('Server started')
```

Fastest Node.js HTTP framework (benchmarks #1 2024)

04. HOOKS

Workflow

```
fastify.addHook('onRequest', async (req, repl...
// auth check
})
fastify.addHook('preHandler', async (req, rep...
// rate limiting
})
fastify.addHook('onSend', async (req, reply, ...
```

Hooks: onRequestpreParsingpreValidationpreHandleronSendonResponse

02. ROUTES

Architecture

```
fastify.get('/users', async (request, reply) ...
return { users: [] } // auto-serialized
})
fastify.post('/users', {
  schema: { body: userSchema, response: { 201: ...
}, async (req, reply) => {
  reply.status(201).send(newUser)
})
```

05. PLUGINS

CRUD API

```
import fp from 'fastify-plugin'
const myPlugin = fp(async (fastify, opts) => {
  fastify.decorate('db', new DB(opts))
})
fastify.register(myPlugin, { uri: process.env...
fastify.register(require('@fastify/cors'))
fastify.register(require('@fastify/jwt'), { s...
fp() makes plugin available across encapsulation boundaries
```

03. SCHEMA VALIDATION

YAML / Config

```
const schema = {
  body: {
    type: 'object',
    required: ['name', 'email'],
    properties: {
      name: { type: 'string', minLength: 2 },
      email: { type: 'string', format: 'email' }
    }
  }
}
```

JSON Schema via Ajv, also boosts serialization 2x

06. DECORATORS

Integration

```
fastify.decorate('config', { port: 3000 })
fastify.decorateRequest('user', null)
fastify.decorateReply('sendError', function(c...
this.status(code).send({ error: msg })
})
// In handler:
request.user / reply.sendError(401, 'Unauth...
```



SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. REQUEST & REPLY

Hardening

```
request.body / request.params / request.query
request.headers / request.ip / request.id
reply.send(data) // infers Content-Type
reply.status(404).send({ error: 'Not found' })
reply.type('text/html').send('<h1>Hi</h1>')
reply.redirect('/new-url')
reply.header('X-Custom', 'value')
```

10. COMMON PLUGINS

Unit Tests

```
@fastify/jwt      JWT auth
@fastify/cors     CORS headers
@fastify/static   static files
@fastify/swagger  OpenAPI docs
@fastify/rate-limit rate limiting
@fastify/compress gzip/deflate
@fastify/sensible httpErrors helpers
```

08. ERROR HANDLING

Observability

```
fastify.setErrorHandler(async (error, request...
request.log.error(error)
reply.status(error.statusCode ?? 500).send({
error: error.message
})
})
throw fastify.httpErrors.notFound('User not f...
// Requires @fastify/sensible plugin
```

11. COMMON GOTCHAS

Commands

Schema must use `additionalProperties:false` or extra props pass
Decorated properties must be initialized in `decorateRequest/Reply`
Plugin encapsulation: `fp()` needed to share across contexts
`reply.send()` after `async` handler = duplicate response error
JSON schema `'format'` requires `ajv-formats` plugin

09. LOGGING

Optimization

```
const fastify = Fastify({
  logger: {
    level: 'info',
    transport: { target: 'pino-pretty' }
  }
})
request.log.info({ userId: id }, 'Fetching us...
fastify.log.error({ err }, 'Fatal error')
```

Built on Pino - very fast structured logging